

ESCUELA POLITÉCNICA NACIONAL

Facultad de Sistemas

Ingeniería en Ciencias de la Computación

Nombre del profesor: Mg . Marco Aguirre

Nombre: Justin Cusquillo

Curso: GR1CC

Fecha de entrega: 25 de octubre del 2026

Laboratorio 3

"E SCIENTIA HOMINIS SALUS"

1. Introducción

Este laboratorio analiza prácticamente la creación, sincronización y reemplazo de procesos en Linux utilizando scripts en Python. Mediante herramientas de monitoreo del sistema, se examina el ciclo de vida y la jerarquía de los procesos concurrentes, identificando cómo la falta de sincronización entre padres e hijos genera estados anómalos como procesos huérfanos y zombis

2. Objetivos

-Comprender la jerarquía y el ciclo de vida de los procesos en un entorno Linux mediante el uso de la biblioteca nativa de Python

-Aplicar mecanismos de sincronización entre procesos concurrentes para evitar la retención innecesaria de recursos

-Evaluar el comportamiento del sistema operativo frente a la terminación asíncrona de tareas, distinguiendo experimentalmente las causas que generan procesos en estado huérfano o zombi

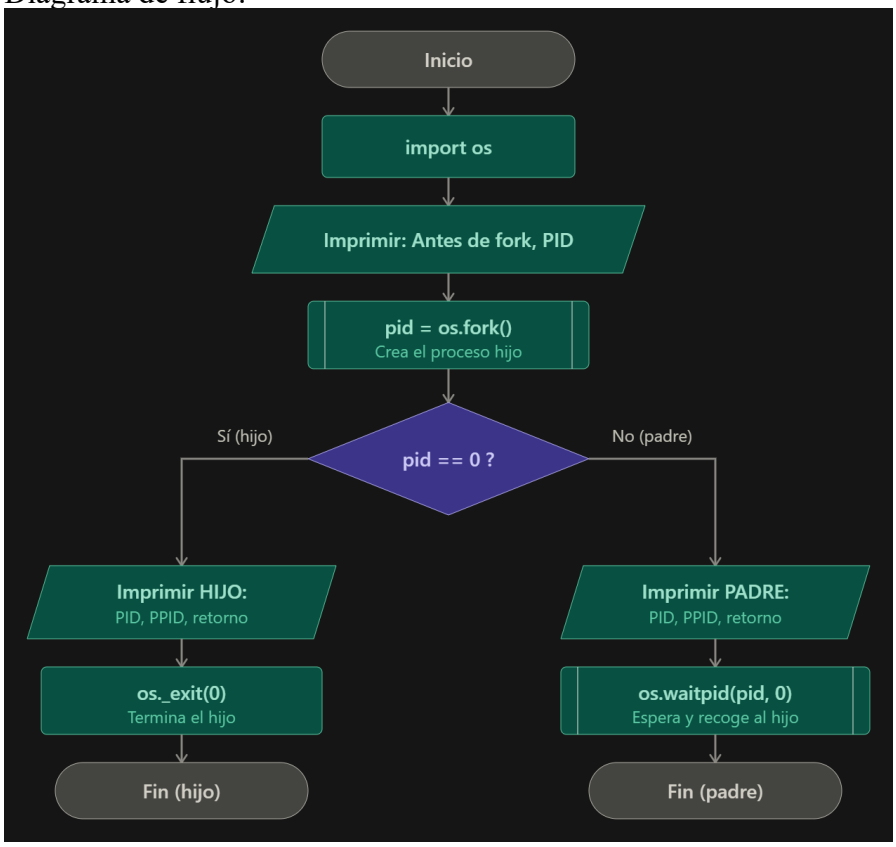
3. Metodología y procedimiento

Actividad 1: creación de un proceso con fork()

Pseudicodigo:

```
INICIO
    IMPORTAR módulo os
    MOSTRAR "Antes de fork:" y PID del proceso actual
    pid ← CREAR_PROCESO_HIJO()        // os.fork()
    SI pid = 0 ENTONCES
            MOSTRAR "HIJO:" PID, PPID y valor de retorno (pid)
            TERMINAR_INMEDIATAMENTE(0)    // os._exit(0)
    SINO
            MOSTRAR "PADRE:" PID, PPID y valor de retorno (pid)
            ESPERAR_AL_HIJO(pid)          // os.waitpid(pid, 0): recoge al hijo
    FIN SI
FIN
```

Diagrama de flujo:



Código:

```
 GNU nano 8.7.1
import os

print(f"Antes de fork: PID={os.getpid()}", flush=True)
pid = os.fork()
if pid == 0:
    print(f"HIJO: PID={os.getpid()}, PPID={os.getppid()}, "
          f"returno={pid}", flush=True)
    os._exit(0)
else:
    print(f"PADRE: PID={os.getpid()}, PPID={os.getppid()}, "
          f"retorno={pid}", flush=True)
    os.waitpid(pid,0) #Recoger al hijo antes de terminar.
```

1. Ejecute el programa varias veces.

```
justin@ubuntu-server:~/laboratorio_procesos$ python3 01_fork_basico.py
Antes de fork: PID=1792
PADRE: PID=1792, PPID=1460, retorno=1793
HIJO: PID=1793, PPID=1792, returno=0
justin@ubuntu-server:~/laboratorio_procesos$ python3 01_fork_basico.py
Antes de fork: PID=1803
PADRE: PID=1803, PPID=1460, retorno=1804
HIJO: PID=1804, PPID=1803, returno=0
justin@ubuntu-server:~/laboratorio_procesos$ python3 01_fork_basico.py
Antes de fork: PID=1814
PADRE: PID=1814, PPID=1460, retorno=1815
HIJO: PID=1815, PPID=1814, returno=0
justin@ubuntu-server:~/laboratorio_procesos$ python3 01_fork_basico.py
Antes de fork: PID=1826
HIJO: PID=1829, PPID=1826, returno=0
PADRE: PID=1826, PPID=1460, retorno=1829
```

2. Identifique el PID del padre, el PID del hijo y el PPID observado por el hijo (ps o con htop).

```
justin@ubuntu-server:~/laboratorio_procesos$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
justin      1460    1102  0 14:43 tty1     00:00:00 -bash
justin      1957    1460  0 15:22 tty1     00:00:00 ps -f
justin@ubuntu-server:~/laboratorio_procesos$ _
```

3. Compare el valor retornado por os.fork() en el padre y en el hijo.

```
, retorno=2016
 returno=0
```

4. Determine si el orden de impresión es siempre el mismo entre ejecuciones. Aunque puede repetirse, no está garantizado. La llamada final a waitpid() recoge al hijo; no fija el orden de los mensajes anteriores

Tras realizar varias ejecuciones, se comprueba que el orden de impresión de los mensajes entre el proceso padre y el hijo puede variar, pero lo que podemos observar este siempre el hijo retorna con 0 y en el caso del padre retorna diferentes valores

Actividad 2: sincronización del padre con wait()

Pseudicodigo:

```
INICIO
    IMPORTAR módulos os, time
    pid ← CREAR_PROCESO_HIJO()  // os.fork()
    SI pid = 0 ENTONCES
            MOSTRAR "HIJO:" PID y PPID
            ESPERAR 3 segundos // time.sleep(3)
            MOSTRAR "HIJO: termino con codigo 7"
            TERMINAR_INMEDIATAMENTE(7) // os._exit(7)
    SINO
            MOSTRAR "PADRE:" PID y "espero a" pid
            (terminado, estado) ← ESPERAR_HIJO()   // os.wait():
            codigo ← DECODIFICAR_ESTADO(estado)    // os.waitstatus_to_exitcode
```

MOSTRAR "PADRE: continuo", hijo = terminado, estado bruto = estado, codigo = codigo
FIN SI
FIN

Diagrama de flujo:



Código:

```
  GNU nano 8.7.1
import os
import time

pid = os.fork()
if pid == 0:
     print(f"HIJO: PID{os.getpid()}, PPID={os.getppid()}",
           flush=True)
     time.sleep(3)
     print("HIJO: termino con codigo 7", flush=True)
     os._exit(7)
else:
     print(f"PADRE: PID={os.getpid()}, espera a {pid}",
           flush=True)
     terminado, estado = os.wait()
     codigo = os.waitstatus_to_exitcode(estado)
     print(f"PADRE: continuo; hijo={terminado}, "
           f"estado bruto={estado}, codigo={codigo}", flush=True)
```

1. Identifique el momento en que el padre queda bloqueado esperando al hijo.

```
PADRE: PID=2482, espera a 2483
HIJO: PID2483, PPID=2482
```

2. Compruebe que el padre continúa únicamente después de que el hijo haya finalizado.

```
HIJO: termino con codigo 7
PADRE: continuo; hijo=2483, estado bruto=1792, codigo=7
```

3. Identifique el código de salida devuelto por el hijo.

```
 hijo=2483, estado bruto=1792, codigo=7
```

4. Modifique el tiempo de espera del hijo y repita la ejecución.

```
import os
import time

pid = os.fork()
if pid == 0:
    print(f"HIJO: PID{os.getpid()}, PPID={os.getppid()}",
          flush=True)
    time.sleep(10)
    print("HIJO: termino con codigo 7", flush=True)
    os._exit(7)
else:
    print(f"PADRE: PID={os.getpid()}, espera a {pid}",
          flush=True)
    terminado, estado = os.wait()
    codigo = os.waitstatus_to_exitcode(estado)
    print(f"PADRE: continuo; hijo={terminado}, "
          f"estado bruto={estado}, codigo={codigo}", flush=True)

justin@ubuntu-server:~/laboratorio_procesos$ python3 02_fork_wait
PADRE: PID=2514, espera a 2515
HIJO: PID2515, PPID=2514
HIJO: termino con codigo 7
PADRE: continuo; hijo=2515, estado bruto=1792, codigo=7
```

Se modificó el tiempo de espera del proceso hijo de 3 a 10 segundos mediante la instrucción time.sleep(5) y luego se ejecutó nuevamente el programa, lo que se observó es que el proceso padre permaneció esperando hasta que el hijo terminó su ejecución. Al finalizar el hijo, el padre continuó y mostró el código de salida 7

Actividad de consulta

Investigue qué señal recibe el proceso padre cuando un proceso hijo termina su ejecución. Explique qué es la señal SIGCHLD, en qué situaciones se genera y cómo se relaciona con la función os.wait().

-Señal SIGCHLD: Es una interrupción asíncrona enviada por el kernel de Linux para notificar a un proceso supervisor sobre los cambios de estado en sus procesos dependientes

-Situaciones de generación: Se desencadena de forma automática cuando un proceso hijo finaliza su ejecución se detiene temporalmente o reanuda su actividad después de haber sido suspendido

-Relación con os.wait(): La función os.wait() suspende la ejecución del proceso padre, manteniéndolo inactivo hasta que recibe la señal SIGCHLD y al interceptar esta señal, os.wait() despierta al padre, recolecta el código de salida del hijo terminado y elimina su registro de la tabla de procesos del sistema operativo, garantizando que el hijo no se convierta en un proceso zombi

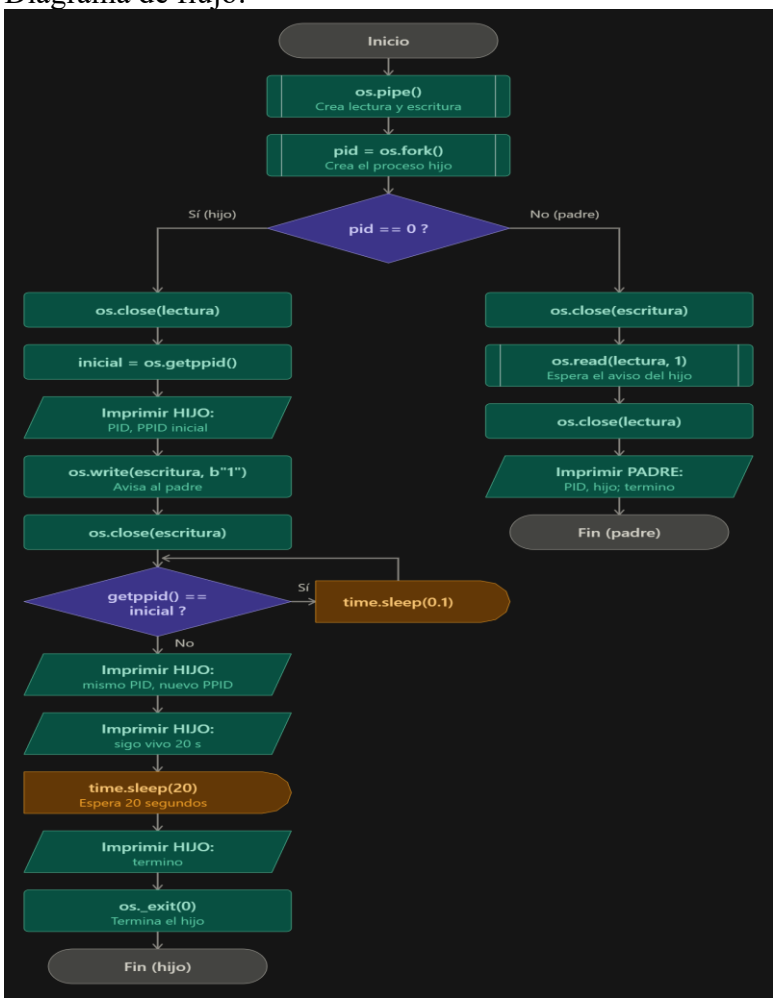
Actividad 3: proceso huérfano

Pseudicodigo:

```
INICIO
  IMPORTAR módulos os, time
  (lectura, escritura) ← CREAR_CANAL()// os.pipe()
  pid ← CREAR_PROCESO_HIJO()// os.fork()
  SI pid = 0 ENTONCES
    CERRAR(lectura)
    inicial ← PPID_ACTUAL()// os.getppid()
    MOSTRAR "HIJO:" PID y PPID inicial
    ESCRIBIR(escritura, "1")
    CERRAR(escritura)
    MIENTRAS PPID_ACTUAL() = inicial HACER
      ESPERAR 0.1 segundos // time.sleep(0.1)
    FIN MIENTRAS
    MOSTRAR "HIJO:" mismo PID y nuevo PPID
    MOSTRAR "HIJO: sigo vivo 20 s para observar con ps"
    ESPERAR 20 segundos    // time.sleep(20)
    MOSTRAR "HIJO: termino"
    TERMINAR_INMEDIATAMENTE(0  // os._exit(0)
  SINO
    CERRAR(escritura)
```

LEER(lectura, 1 byte)
CERRAR(lectura)
MOSTRAR "PADRE:" PID, hijo = pid; termino
FIN SI
FIN

Diagrama de flujo:



Código:

```
import os
import time

# El canal garantiza que el hijo mida su PPID antes
# de que el padre termine. No recoge estados de salida.
lectura, escritura = os.pipe()
pid = os.fork()
if pid == 0:
    os.close(lectura)
    inicial = os.getppid()
    print(f"HIJO: PID={os.getpid()}, PPID inicial={inicial}",
          flush=True)
    os.write(escritura,b"1")
    os.close(escritura)
    while os.getppid() == inicial:
        time.sleep(0.1)
    print(f"HIJO: mismo PID={os.getpid()},"
          f"nuevo PPID={os.getppid()}", flush=True)
    print("HIJO: sigo vivo 20 s para observar con ps ", flush=True)
    time.sleep(20)
    print("HIJO: termino", flush=True)
    os._exit(0)
else:
    os.close(escritura)
    os.read(lectura, 1)
    os.close(lectura)
    print(f"PADRE: PID={os.getpid()}, hijo={pid}; "
          "termino sin wait", flush=True)
    os._exit(0)
```

1. Ejecute el programa desde una terminal.

```
justin@ubuntu-server:~/laboratorio_procesos$ python3 03_huerfano.py
HIJO: PID=2896, PPID inicial=2895
PADRE: PID=2895, hijo=2896; termino sin wait
justin@ubuntu-server:~/laboratorio_procesos$ HIJO: mismo PID=2896,nuevo PPID=1
HIJO: sigo vivo 20 s para observar con ps
```

2. Observe el PPID del hijo al iniciar (ps o htop requerido).

```
HIJO: PID=3416, PPID inicial=3415
```

3. Mientras el hijo duerme, identifique el proceso mediante una herramienta de monitoreo del sistema (ps o htop requerido).

```
justin@ubuntu-server:~$ ps -o pid,ppid,stat,cmd -p 3586
    PID    PPID STAT CMD
   3586       1 S    python3 03_huerfano.py
```

4. Observe nuevamente el PPID mostrado por el hijo después de que el padre haya terminado.

```
HIJO: mismo PID=3710,nuevo PPID=1
```

5. Determine qué proceso adoptó al hijo en su entorno Linux (ps o htop requerido).

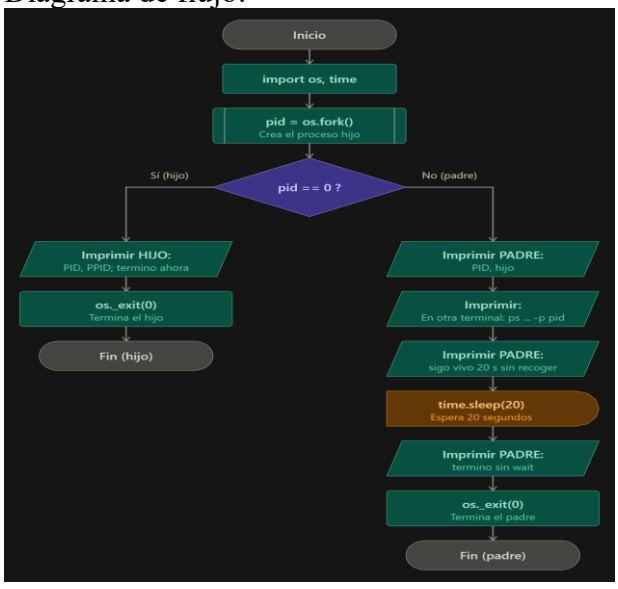
```
justin@ubuntu-server:~$ ps -p 1 -o pid,ppid,cmd
    PID    PPID CMD
      1       0 /usr/lib/systemd/systemd --switched-root --system --deserialize=51
```

Actividad 4: proceso zombi al no utilizar wait()

Pseudicodigo:

```
INICIO
   IMPORTAR módulos os, time
   pid ← CREAR_PROCESO_HIJO()// os.fork()
   SI pid = 0 ENTONCES
      MOSTRAR "HIJO:" PID, PPID; termino ahora
      TERMINAR_INMEDIATAMENTE(0)  // os._exit(0)
   SINO
      MOSTRAR "PADRE:" PID, hijo = pid
      MOSTRAR "En otra terminal: ps -o pid,ppid,stat,cmd -p" pid
      MOSTRAR "PADRE: sigo vivo 20 s sin recoger al hijo"
      ESPERAR 20 segundos   // time.sleep(20)
      MOSTRAR "PADRE: termino sin wait"
      TERMINAR_INMEDIATAMENTE(0) // os._exit(0)
   FIN SI
FIN
```

Diagrama de flujo:



Código:

```
  GNU nano 8.7.1
import os
import time

pid = os.fork()
if pid == 0:
     print(f"HIJO: PID={os.getpid()}, PPID={os.getppid()}; "
           "termino ahora", flush=True)
     os._exit(0)
else:
    print(f"PADRE: PID={os.getpid()}, hijo={pid}", flush=True)
    time.sleep(5) #Mantener al padre activo inicialmente
    terminado, estado = os.waitpid(pid,0)
    print(f"PADRE: hijo{terminado} recodigo con waitpid()", flush=True)
    print("PADRE: sigo vivo 10 s despues de waitpid()", flush=True)
    time.sleep(10)
    print("PADRE: termino", flush=True)
    os._exit(0)
```

1. Ejecute el programa.

```
justin@ubuntu-server:~/laboratorio_procesos$ python3 04_zombie.py
PADRE: PID=4149, hijo=4150
HIJO: PID=4150, PPID=4149; termino ahora
En otra terminal: ps -0 pid,ppid,stat,cmd -p 4150
PADRE: sigo vivo 20 s sin recoger al hijo
```

2. Durante los 20 segundos en que el padre permanece activo, localice el proceso hijo mediante ps u otra herramienta equivalente.

```
justin@ubuntu-server:~$ ps -o pid,ppid,stat,cmd -p 4072
    PID    PPID STAT CMD
   4072    4071 Z+   [python3] <defunct>
justin@ubuntu-server:~$
```

3. Identifique el estado del hijo y compruebe si aparece marcado como Z o <defunt >.

```
 [python3] <defunct>
:~$
```

4. Espere a que el padre termine y vuelva a verificar si el proceso zombi continúa apareciendo. Normalmente el adoptante lo recolectará; en un contenedor mal configurado esto puede demorarse.

```
justin@ubuntu-server:~$ ps -o pid,ppid,stat,cmd -p 4072
    PID    PPID STAT CMD
justin@ubuntu-server:~$
```

5. En una segunda ejecución, añada os.waitpid(pid, 0) inmediatamente después de la pausa del padre y agregue otra pausa de 10 segundos. Compruebe que el zombi desaparece mientras el padre sigue vivo. Documente también el pseudocódigo y el diagrama de esta variante.

```
  GNU nano 8.7.1
import os
import time

pid = os.fork()
if pid == 0:
     print(f"HIJO: PID={os.getpid()}, PPID={os.getppid()}; "
           "termino ahora", flush=True)
     os._exit(0)
else:
    print(f"PADRE: PID={os.getpid()}, hijo={pid}", flush=True)
    time.sleep(5) #Mantener al padre activo inicialmente
    terminado, estado = os.waitpid(pid,0)
    print(f"PADRE: hijo{terminado} recodigo con waitpid()", flush=True)
    print("PADRE: sigo vivo 10 s despues de waitpid()", flush=True)
    time.sleep(10)
    print("PADRE: termino", flush=True)
    os._exit(0)
```

```
justin@ubuntu-server:~/laboratorio_procesos$ python3 04_zombie.py
PADRE: PID=4622, hijo=4623
HIJO: PID=4623, PPID=4622; termino ahora
PADRE: hijo4623 recodigo con waitpid()
PADRE: sigo vivo 10 s despues de waitpid()
PADRE: termino
justin@ubuntu-server:~/laboratorio_procesos$
```

```
justin@ubuntu-server:~$ ps -o pid,ppid,stat,cmd -p 4622
    PID    PPID STAT CMD
   4622    1460 S+   python3 04_zombie.py
justin@ubuntu-server:~$ _
```

Actividad de consulta

Explique por qué el proceso hijo puede permanecer como zombi aunque ya haya terminado su ejecución. Relacione este comportamiento con la necesidad de que el padre recoja el estado de salida mediante wait() o waitpid()

Un proceso hijo puede permanecer como zombi cuando ya terminó su ejecución, pero su proceso padre todavía no ha recogido su estado de salida. El sistema operativo conserva información del hijo, como su PID y su código de terminación, para que el padre pueda consultarla. Para liberar esa información, el padre debe utilizar wait() o waitpid()

Actividad 5: creación de carpetas mediante fork(), exec() y waitpid()

Pseudicodigo:

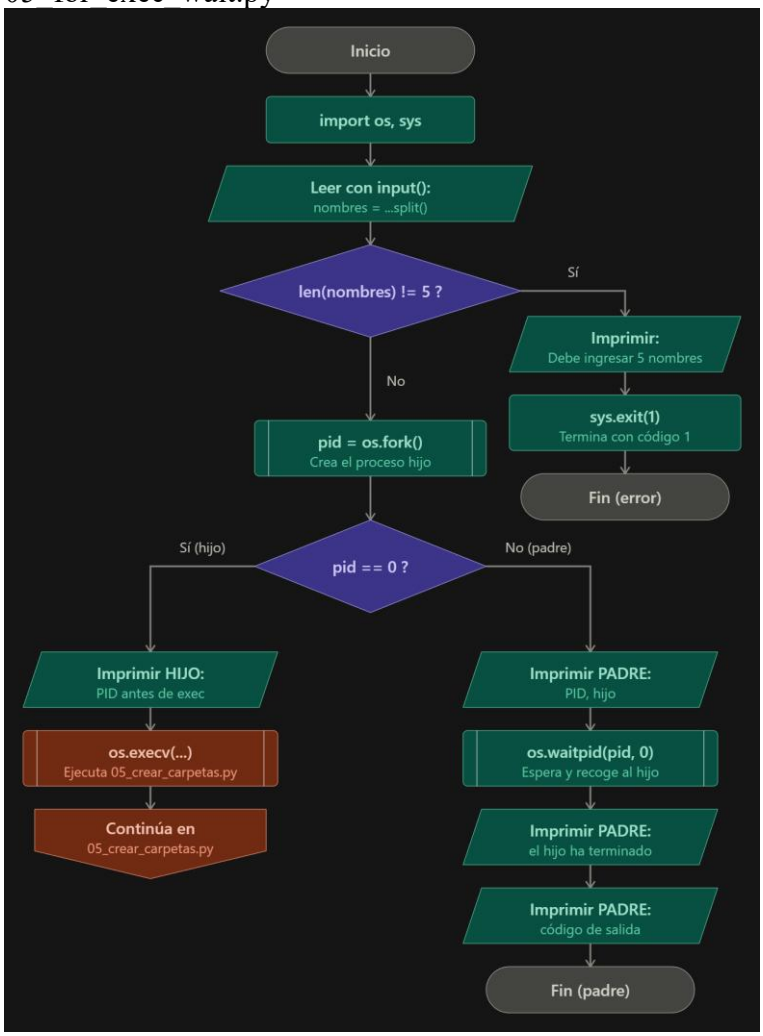
05_for_exec_wait.py

```
INICIO
   IMPORTAR módulos os, sys
   nombres ← LEER_ENTRADA("Ingrese cinco nombres de carpetas...")
            y DIVIDIR por espacios  // input().split()
   SI longitud(nombres) ≠ 5 ENTONCES
      MOSTRAR "Debe ingresar exactamente cinco nombres."
      TERMINAR(1)  // sys.exit(1)
   FIN SI
   pid ← CREAR_PROCESO_HIJO()    // os.fork()
   SI pid = 0 ENTONCES     // Proceso HIJO
      MOSTRAR "HIJO: PID antes de exec" y PID
      EJECUTAR_PROGRAMA(python, "05_crear_carpetas.py", nombres)// os.execv: reemplaza al hijo
   SINO
      MOSTRAR "PADRE:" PID, hijo = pid
      (terminado, estado) ← ESPERAR_HIJO(pid) // os.waitpid(pid, 0)
      MOSTRAR "PADRE: el hijo" terminado "ha terminado."
      MOSTRAR "PADRE: código de salida =" DECODIFICAR(estado)
   FIN SI
FIN
```

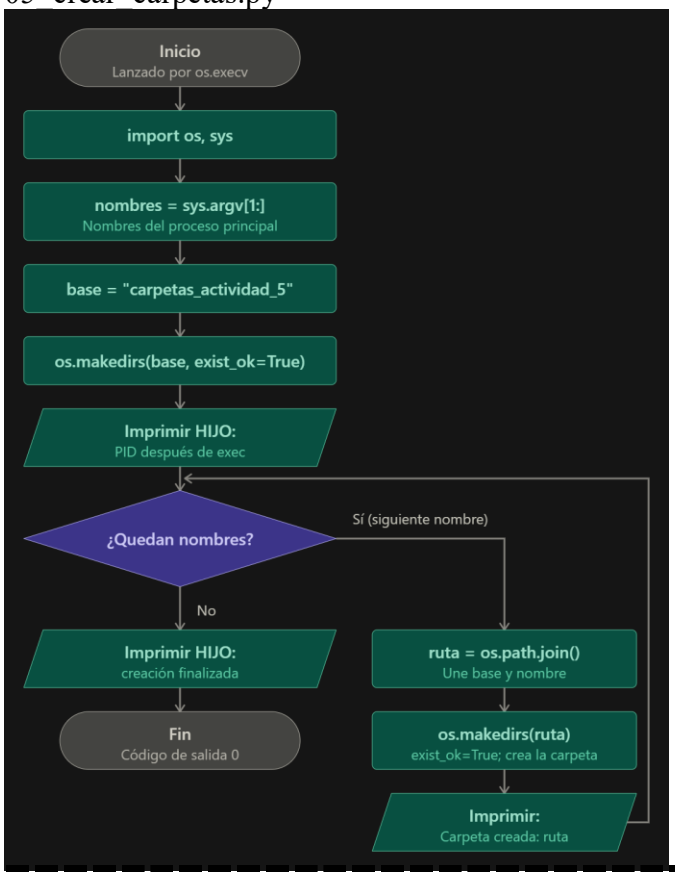
05_crear_carpetas.py

```
INICIO                // lanzado por os.execv
   IMPORTAR módulos os, sys
   nombres ← ARGUMENTOS_RECIBIDOS()   // sys.argv[1:]
   base ← "carpetas_actividad_5"
   CREAR_CARPETA(base, sin error si ya existe)   // os.makedirs(base, exist_ok=True)
   MOSTRAR "HIJO: PID después de exec" y PID
   PARA CADA nombre EN nombres HACER
      ruta ← UNIR(base, nombre)    // os.path.join(base, nombre)
      CREAR_CARPETA(ruta, sin error si ya existe)
      MOSTRAR "Carpeta creada:" ruta
   FIN PARA
   MOSTRAR "HIJO: creación de carpetas finalizada."
 FIN
```

Diagrama de flujo:
05_for_exec_wait.py



05_crear_carpetas.py



Código:

05_for_exec_wait.py

```
import os
import sys

nombres = input("Ingrese cinco nombres de carpetas seprados por espacios: ").split()

if len(nombres) != 5:
    print("Debe ingresar exactamente cinco nombres. ")
    sys.exit(1)


pid = os.fork()

if pid == 0:
    print(f"HIJO: PID antes de exec={os.getpid()}", flush=True)

    os.execv(sys.executable,[sys.executable, "05_crear_carpetas.py"] + nombres)

else:
    print(f"PADRE: PID={os.getpid()}, hijo={pid}", flush=True)
    terminado, estado  = os.waitpid(pid, 0)

    print(f"PADRE: el hijo{terminado} ha terminado", flush=True)
    print(f"PADRE: codigo de salida={os.waitstatus_to_exitcode(estado)}")
```

05_crear_carpetas.py

```
import os
import sys

nombres = sys.argv[1:]

base = "carpetas_actividad_5"
os.makedirs(base, exist_ok=True)

print(f"HIJO: PID despues de exec={os.getpid()}", flush=True)

for nombre in nombres:
    ruta = os.path.join(base,nombre)
    os.makedirs(ruta, exits_ok=True)
    print(f"Carpeta creada: {ruta}", flush=True)

print("HIJO: creacion de carpetas finalizada", flush=True)
```

1. Ejecute el programa principal e ingrese al menos cinco nombres de carpetas diferentes. Compruebe que fueron creadas correctamente.

```
justin@ubuntu-server:~/laboratorio_procesos$ python3 05_for_exec_wait.py
Ingrese cinco nombres de carpetas seprados por espacios: carpeta1 carpeta2 carpeta3 carpeta4 carpeta5
```

2. Identifique el PID del proceso padre y el PID del proceso hijo.

```
PADRE: PID=4904, hijo=4905
```

3. Compare el PID del hijo antes de llamar a exec() con el PID que muestra el segundo programa Python. Explique el resultado.

```
HIJO: PID antes de exec=4905
HIJO: PID despues de exec=4905
```

El PID del proceso hijo antes de llamar a exec() es el mismo que muestra el segundo programa Python. Esto sucede porque exec() reemplaza el programa que ejecuta el proceso hijo, pero no crea un proceso nuevo y por lo tanto, el proceso conserva su PID durante la ejecución del segundo programa

4. Identifique qué proceso solicita los nombres de las carpetas y qué proceso realiza su creación.

El proceso padre solicita al usuario los nombres de las carpetas mediante la entrada de datos y después, utiliza fork() para crear un proceso hijo. El proceso hijo ejecuta el segundo programa Python mediante exec(), y este programa se encarga de crear las carpetas solicitadas

5. Explique qué sucede con el proceso padre mientras el hijo está creando las carpetas.

El proceso padre utiliza waitpid() para esperar a que el proceso hijo termine de crear las carpetas.

Mientras el hijo realiza esta tarea, el padre permanece bloqueado esperando su finalización ya cuando el hijo termina, el padre continúa su ejecución y muestra el mensaje correspondiente

6. Modifique el segundo programa para que el tiempo de espera entre la creación de cada carpeta sea de cinco segundos. Observe el comportamiento de ambos procesos.

```
GNU nano 6.7.1
import os
import sys
import time
nombres = sys.argv[1:]

base = "carpetas_actividad_5"
os.makedirs(base, exist_ok=True)

print(f"HIJO: PID despues de exec={os.getpid()}", flush=True)

for nombre in nombres:
    ruta = os.path.join(base,nombre)
    os.makedirs(ruta, exist_ok=True)
    print(f"Carpeta creada: {ruta}", flush=True)

    time.sleep(5)

print("HIJO: creacion de carpetas finalizada", flush=True)
```

```
Ingrese cinco nombres de carpetas seprados por espacios: carpeta1 carpeta2 carpeta3 carpeta4 carpeta5
PADRE: PID=5098, hijo=5099
HIJO: PID antes de exec=5099
HIJO: PID despues de exec=5099
Carpeta creada: carpetas_actividad_5/carpeta1
Carpeta creada: carpetas_actividad_5/carpeta2
```

Al ejecutar el programa, el proceso hijo crea una carpeta y espera cinco segundos antes de continuar con la siguiente. Mientras tanto, el proceso padre permanece esperando mediante waitpid(). Cuando el hijo termina de crear todas las carpetas, el padre continúa su ejecución.

7. Ejecute nuevamente el programa utilizando nombres de carpetas diferentes y verifique los resultados mediante comandos de Linux.

```
justin@ubuntu-server:~/laboratorio_procesos$ ls -l carpetas_actividad_5
total 40
drwxrwxr-x 2 justin justin 4096 Oct 23 18:37 Dinamica
drwxrwxr-x 2 justin justin 4096 Oct 23 18:38 EDA
drwxrwxr-x 2 justin justin 4096 Oct 23 18:38 Estatica
drwxrwxr-x 2 justin justin 4096 Oct 23 18:38 Progra2
drwxrwxr-x 2 justin justin 4096 Oct 23 18:38 SO
drwxrwxr-x 2 justin justin 4096 Oct 23 17:26 carpeta1
drwxrwxr-x 2 justin justin 4096 Oct 23 17:26 carpeta2
drwxrwxr-x 2 justin justin 4096 Oct 23 17:26 carpeta3
drwxrwxr-x 2 justin justin 4096 Oct 23 17:26 carpeta4
drwxrwxr-x 2 justin justin 4096 Oct 23 17:26 carpeta5
```

8. Explique qué sucedería si el proceso padre no utilizara waitpid() y terminara antes de que el hijo completara su trabajo

Si el proceso padre no utilizara waitpid() y terminara antes que el hijo, este último podría continuar ejecutándose de manera independiente, el proceso hijo se convertiría en un proceso huérfano y sería adoptado por otro proceso del sistema operativo. Como consecuencia, el padre no esperaría la finalización del trabajo del hijo ni recogería su estado de salida, lo que podría ocasionar problemas de sincronización y control de la ejecución.

Actividad 6: comparación experimental

Caso	¿Quién termina primero?	Uso de wait	Comportamiento observado
Padre espera al hijo	El hijo termina su trabajo y luego el padre continúa y termina	Sí, el padre utiliza wait() o waitpid()	El padre espera a que el hijo finalice y recoge su estado de terminación
Proceso huérfano	El padre termina primero; el hijo sigue ejecutándose	No se utiliza wait() antes de que termine el padre	El hijo continúa ejecutándose y pasa a ser adoptado por otro proceso del sistema

Proceso zombi	El hijo termina primero, pero el padre sigue activo	El padre todavía no ha utilizado wait() para recoger al hijo	El hijo ya terminó, pero permanece como zombi hasta que el padre recoge su estado

4. Preguntas de análisis

Responda brevemente con sus propias palabras:

1. ¿Qué diferencia existe entre el PID retornado por fork() en el padre y el valor recibido en el hijo?

En el proceso padre, la función fork() retorna el PID real del nuevo proceso hijo creado; en el proceso hijo, la función retorna el valor 0

2. ¿Qué problema evita el uso de wait() después de que un hijo termina?

Evita la acumulación de procesos zombis al permitir que el sistema operativo recoja el estado de terminación del hijo y libere sus recursos de la tabla de procesos.

3. ¿Cuál es la diferencia entre un proceso huérfano y un proceso zombi?

Un proceso huérfano surge cuando el padre termina su ejecución pero el hijo continúa activo. Un proceso zombi ocurre cuando el hijo ya terminó, pero el padre sigue activo y no ha recogido su estado mediante wait()

4. ¿No utilizar wait() siempre produce un proceso huérfano? Justifique su respuesta.

Omitir wait() no convierte a un hijo en huérfano de forma automática. Si el hijo termina antes y no hay wait(), se convierte en zombi; la orfandad solo ocurre si el padre muere antes que el hijo

5. ¿Qué ocurre con el PID cuando un proceso ejecuta exec()?

El PID se conserva sin cambios, dado que exec() reemplaza el programa que ejecuta el proceso, pero mantiene su identidad en el sistema

6. ¿Cuál es la diferencia conceptual entre fork() y exec()?

fork() crea un nuevo proceso clonando al padre. Por su parte, exec() no crea procesos nuevos, sino que sobrescribe el proceso actual para ejecutar un binario diferente

7. ¿Por qué la combinación fork() + exec() + wait() es útil para implementar un shell?

Un shell emplea fork() para crear un subproceso, utiliza exec() dentro de ese hijo para ejecutar el comando solicitado por el usuario, y usa wait() en el padre para pausar la terminal hasta que el comando termine

8. ¿Qué diferencia existe entre wait() y waitpid()?

wait() bloquea al padre hasta que finaliza cualquier hijo, mientras que waitpid() le permite al padre esperar específicamente la finalización de un proceso hijo en particular indicando su PID

5. Conclusión

La práctica demostró empíricamente que sincronizar procesos mediante wait() es indispensable para recolectar estados de salida y evitar la acumulación de procesos zombis en el sistema. Asimismo, se comprobó que cuando un proceso supervisor termina prematuramente, el sistema operativo interviene automáticamente para adoptar a los procesos huérfanos resultantes.

6. Anexos:

Anexo: Uso de Inteligencia Artificial Generativa

Elemento	Descripción del Estudiante
Modelo Utilizado	Google Gemini.
Prompt	"Podrías guiarme paso a paso para comprender los conceptos de este laboratorio sobre procesos en Linux, de manera que pueda entender cómo funciona y resolverlo por mi cuenta?".
Resultado Obtenido	Explicación conceptual sobre la jerarquía de procesos, definición de PIDs, estados de procesos (huérfanos y zombis), y orientación teórica sobre el uso de comandos de monitoreo en Linux.
Validación	Se contrastó la guía teórica proporcionada por la IA ejecutando los scripts en la máquina virtual con Ubuntu Server, verificando empíricamente el comportamiento de los procesos con las herramientas del sistema para redactar el análisis final

[1]Python Software Foundation, "os — Miscellaneous operating system interfaces," Python 3 Documentation, 2026. [Online]. Available: https://docs.python.org/3/library/os.html
[2] M. Kerrisk, Ed., "Linux man-pages project," Linux kernel API documentation. [Online]. Available: https://man7.org/linux/man-pages/
[3] Canonical, "Ubuntu Server documentation," Ubuntu Documentation. [Online]. Available: https://documentation.ubuntu.com/server/.
[4] J. Torres, "Sistemas Operativos," Universidad de La Laguna, OpenCourseWare (OCW). [Online]. Available: https://ull-esit-sistemas-operativos.github.io/ssoo-apuntes/ocw/
[5] M. Aguirre, Laboratorio IV: Creación, sincronización y ejecución de procesos en Python. Escuela Politécnica Nacional, Sistemas Operativos, período 2026-B, Quito, Ecuador, 2026.